SOFTWARE TESTING AND EVALUATION

Application of Verification and Validation Techniques

A Practical Assignment Report on

Taskify : Job Application Tracker

Name	Nirajkumar Patel
Roll Number	BT22CSE179
Subject	Software Testing and Evaluation
Submitted To	Dr.Vrinda Yadav
Date	14 March 2026
Project Repository	github.com/npxpatel/Taskify

1. Introduction

Software Testing and Evaluation is a foundational discipline in software engineering that concerns itself with the correctness, completeness, and reliability of software systems. According to Yogesh Singh, software testing is the process of executing a program with the intention of finding errors, while verification is a static process that ensures software artifacts are developed correctly without necessarily running the code.

Taskify is a full-stack job application tracker that addresses a real problem faced by active job seekers: the overhead of managing dozens of applications, follow-up tasks, and email templates across disconnected tools. Because the system handles time-sensitive user data — application statuses, due-date reminders, and asynchronous email delivery — failures can lead to missed opportunities, stale reminders not being sent, or loss of application history. Systematic application of Verification and Validation (V&V) techniques is therefore critical.

This report applies the V&V techniques taught up to Chapter 5 of the prescribed textbook on Taskify, demonstrating peer review, walkthroughs, formal inspection, black-box testing, and fault-error-failure analysis on a production-deployed project.

2. Project Overview: Taskify — Job Application Tracker

2.1 Project Description

Taskify is a full-stack web application that helps job seekers consolidate every stage of their job search into one place. The system is built on a Node.js/Express backend with a React 18 + TypeScript frontend, backed by MongoDB for persistence, Redis for per-user caching, and RabbitMQ for asynchronous email delivery.

The application provides four core capabilities: job application tracking (with company logo auto-fetch via Clearbit), a priority-based task manager, reusable email template storage, and automated stale-job reminder emails dispatched through a durable message queue. The backend is deployed on Render (Singapore) and the frontend on Vercel, making it a live, publicly accessible application.

2.2 Objectives of the System

- Allow users to register and log in securely using JWT-based authentication stored in HttpOnly cookies.
- Enable tracking of job applications with company, role, date applied, opening type, and status progression.
- Provide a priority-aware task manager (low / medium / high) with due dates and categorised views.
- Store and seed reusable email templates across multiple categories and types.
- Dispatch automated daily stale-job reminders asynchronously via RabbitMQ, not inline.
- Implement password reset using OTP delivered through the async email worker.
- Cache per-user job lists in Redis with 5-minute TTL and write-through cache invalidation.

- Validate all request bodies using Zod schemas on every route before they reach any controller.

2.3 Technology Stack

Layer	Technology / Component
Frontend	React 18, TypeScript, Vite, Tailwind CSS, shadcn/ui
Backend	Node.js, Express.js
Database	MongoDB (Mongoose ODM)
Cache	Redis — per-user job list cache, 5-minute TTL, write-through invalidation
Message Queue	RabbitMQ — two durable queues: email_notifications and job_reminders
Authentication	JWT in HttpOnly cookie, bcrypt (12 rounds), Zod validation on all inputs
Email	Nodemailer (Gmail SMTP via App Password)
Deployment	Backend: Render (Singapore), Frontend: Vercel

2.4 Major Functional Modules

Module	Description
User Authentication	Registration, login, logout using JWT in HttpOnly cookie; bcrypt 12 rounds; OTP-based password reset.
Job Tracking	CRUD for job applications with company, role, date, opening type, and status. Company logo auto-fetched via Clearbit fuzzy search.
Task Management	To-do list with due dates, priority levels, and descriptions. Dashboard widget for today's tasks. Tabbed view: Overdue / Today / Tomorrow / Upcoming.
Template Storage	Reusable email templates by category and type. Seeds 8 defaults on first login via a dedicated seed endpoint.
Stale Job Reminders	Daily cron at 9 AM publishes a message to RabbitMQ; a separate reminder worker consumes it and emails a list of applications with no update in 14+ days.
Async Email Worker	Consumes email_notifications queue to handle OTP delivery and general emails, keeping all email I/O off the request-response path.
Redis Cache Layer	Caches per-user job lists for 5 minutes. Cache is busted on every write operation (create / update / delete).
Input Validation	All request bodies pass through Zod schemas in the types/ layer before reaching any controller.

3. Verification Techniques Applied

Verification is a static process that checks whether software artifacts are correct, complete, and consistent — without executing the code. The following three techniques were applied to Taskify.

3.1 Peer Review

Description

Peer review is an informal verification technique where peers examine software artifacts to catch defects early. The review for Taskify was carried out on source code, architecture decisions, and project documentation.

Artifacts Reviewed

- Express application structure (app.js, server.js, routing layer)
- Mongoose models: User, Job, Task, Template, Otp
- Authentication middleware and JWT helper utilities
- RabbitMQ worker implementations (emailWorker.js, reminderWorker.js)
- Redis client service and cache invalidation logic
- Zod validation schemas in types/ for all request bodies

Observations

- The stale job reminder cron job was described in the README as running at 9 AM, but no timezone was specified in the cron expression — it was unclear whether this referred to server local time, UTC, or the user's timezone, risking reminders arriving at unexpected hours for users in different regions.
- The OTP expiry was documented as 10 minutes, but no verification existed that the Otp model's TTL index was correctly configured in MongoDB to automatically remove expired documents, potentially leaving stale OTPs in the collection.
- The README noted Redis and RabbitMQ as optional, but there was no documented graceful degradation path for the case where the queue is unavailable but a password reset is attempted — the user would get no OTP email with no meaningful error.
- The company logo fetch via Clearbit had no fallback documented for companies not in the Clearbit dataset — it was unclear whether the frontend showed a placeholder or threw an unhandled error.

Outcome

Peer review identified several design ambiguities. These observations led to documentation improvements: the cron expression was clarified as UTC, the Otp model was confirmed to use a MongoDB TTL index with expireAfterSeconds, graceful degradation for missing queues was explicitly documented, and a placeholder logo fallback was confirmed as present in the frontend component.

3.2 Walkthrough

Description

In a walkthrough, the author traces through the system artifact with peers to verify logical correctness and identify gaps before implementation is complete.

Journeys Walked Through

Journey 1: Registration and First Job Application

Submit empty registration form → validation error from Zod (PASS). Submit mismatched passwords → validation error (PASS). Valid registration → JWT cookie set → redirect to dashboard (PASS). Add job with valid company name → Clearbit logo fetched → job saved to MongoDB → Redis cache busted (PASS). View job list → served from Redis on second load (PASS).

Journey 2: Task Management and Dashboard

Create task with past due date → task created and appears in Overdue tab (PASS). Create task due today → appears in Today tab on dashboard widget (PASS). Mark task complete → task removed from active views (PASS). Create task with no due date — peers raised a question: the dashboard widget only shows today's tasks, so undated tasks had no visibility path. This was noted as a design gap.

Journey 3: Password Reset and OTP Flow

Trigger forgot-password → OTP published to email_notifications queue → worker sends email (PASS). Submit OTP within 10 minutes → password updated (PASS). Submit expired OTP → system rejects it (PASS). Walkthrough revealed that no rate limiting existed on the /api/auth/forgot-password endpoint — an attacker could spam OTP requests, flooding the user's inbox. This was flagged as a security gap.

Findings Summary

- Undated tasks have no visibility path on the dashboard — a design gap noted for future improvement.
- No rate limiting on forgot-password endpoint — security risk identified.
- The seed-defaults endpoint for templates requires no body parameters; walkthrough confirmed it is idempotent (does not duplicate seeds on repeat calls), which was a positive finding.
- Cache invalidation was confirmed to be triggered on all three write operations (POST, PATCH, DELETE) for jobs — no stale-cache scenarios were found.

Outcome

The walkthrough identified one security gap (no rate limiting on OTP requests) and one UX gap (undated tasks invisible on dashboard). Both were documented as known limitations. The cache and queue behaviours were verified as correct.

3.3 Formal Inspection

Formal inspection is the most structured verification technique, applying defined checklists, roles, and exit criteria to detect defects. Inspection was carried out on key source code modules: authentication middleware, job controller, worker implementations, Redis service, and Zod schema definitions.

#	Inspection Item	Result	Location
1	All route handlers use centralized async error handling	**PASS**	middleware/errorHandler.js
2	Passwords hashed with bcrypt at cost 12 before storage	**PASS**	models/User.js
3	JWT stored in HttpOnly cookie (not localStorage)	**PASS**	config/cookieOptions.js
4	All request bodies validated with Zod before controllers	**PASS**	types/ schemas
5	Redis cache invalidated on every write operation	**PASS**	controllers/jobs.js
6	RabbitMQ queues declared as durable	**PASS**	services/rabbitmq.js
7	OTP expiry enforced with MongoDB TTL index	**PASS**	models/Otp.js
8	CORS origins validated against allowlist (not wildcard)	**PASS**	app.js
9	Rate limiting on authentication endpoints	**PARTIAL**	No rate limiter on /forgot-password
10	Worker error handling — failed messages handled gracefully	**PARTIAL**	emailWorker.js — no dead-letter queue defined

Result: 8 PASS 2 PARTIAL 0 FAIL

4. Document Verification

4.1 Software Requirements Specification (SRS) Verification

Taskify has no formal SRS document. Requirements were inferred from the README, the live deployed application, and code inspection.

ID	Requirement	Status	Source
R1	Users can register with email and password; duplicates are rejected	PASS	README / models/User.js
R2	Authentication uses JWT stored in HttpOnly cookie	PASS	README / config/cookieOptions.js
R3	Job applications can be created, updated, and deleted	PASS	README / controllers/jobs.js
R4	Job status follows the pipeline: applied → shortlisted → interviews → selected	PASS	models/Job.js
R5	Per-user job list cached in Redis with 5-minute TTL	PASS	README / services/redis.js
R6	Stale jobs (14+ days without update) trigger daily email reminder	PASS	README / jobs/staleJobReminder.js
R7	Email delivery is asynchronous via RabbitMQ workers	PASS	README / workers/
R8	Password reset uses a 6-digit OTP expiring after 10 minutes	PASS	README / models/Otp.js
R9	Templates are seeded with 8 defaults on first login	PASS	README / controllers/templates.js
R10	Application runs over HTTPS in production	PARTIAL	Handled by Vercel/Render infrastructure

Result: 9/10 fully verified. R10 is satisfied at the infrastructure level by Vercel (frontend) and Render (backend) both enforcing HTTPS termination before requests reach application code — an acceptable and common deployment pattern.

4.2 Software Design Description (SDD) Verification

Design Aspect	Verified Status and Notes
Separation of concerns	PASS — models, controllers, routes, services, workers, jobs, and middleware are cleanly separated with no circular dependencies.
Error handling	PASS — centralized AppError class with a global error handler middleware; all async controllers wrapped in try-catch.

Cache consistency	PASS — Redis invalidated on POST, PATCH, and DELETE for jobs; no stale-read scenarios identified.
Queue durability	PASS — both RabbitMQ queues declared as durable; messages survive broker restart.
Input validation placement	PASS — Zod schemas validated in types/ layer, before any controller logic or database call.
Auth cookie security	PASS — HttpOnly, Secure, SameSite=Strict; no token stored in JavaScript-accessible storage.
Worker independence	PASS — workers run as separate processes and can be started/stopped independently of the main server.
Environment config	PASS — all secrets and URLs in .env; validated at startup; .env excluded from version control.

4.3 Source Code Inspection

A code inspection checklist was applied across the main source modules of the backend:

Checklist Item	Status	Finding
Input Validation	**PASS**	Zod schemas applied to every request body before controller execution; invalid requests rejected at the boundary.
Exception Handling	**PASS**	All async route handlers use try-catch; unhandled errors forwarded to the global error middleware with appropriate HTTP status codes.
Coding Standards	**PASS**	Consistent camelCase for variables/functions, PascalCase for classes/models; consistent file naming throughout.
Boundary Condition Handling	**PASS**	Mongoose schema constraints enforce required fields, enum values, and data types at the model level.
Hardcoded Values	**PASS**	No credentials or URLs hardcoded; all configuration through environment variables with startup-time validation.
Security Headers	**PARTIAL**	Basic CORS configuration present; Helmet.js or explicit security headers (CSP, X-Content-Type-Options) not added.
Cache Invalidation Completeness	**PASS**	Verified that all three write paths (create, update, delete) call the Redis cache invalidation helper.
Worker Error Recovery	**PARTIAL**	Workers catch and log errors per message but no dead-letter queue is configured — persistently failing messages are dropped silently.

5. Validation Techniques Applied

Validation ensures the right product is built by executing test cases against actual user requirements. The following black-box testing techniques were applied to Taskify.

5.1 Equivalence Class Testing

Example: User Registration — Email and Password Fields

The registration endpoint accepts name, email, and password. Inputs are partitioned into valid and invalid equivalence classes.

TC#	Name	Email	Password	Expected Result	Actual Result	Status
EC-1	Valid	user@example.com (valid)	SecurePass1 (≥8 chars)	Account created; JWT cookie set	Account created	**PASS**
EC-2	Valid	notanemail (no @)	SecurePass1	Validation error: Invalid email	Zod error returned	**PASS**
EC-3	Valid	user@ (no domain)	SecurePass1	Validation error: Invalid email	Zod error returned	**PASS**
EC-4	Valid	(empty string)	SecurePass1	Validation error: Email required	Zod error returned	**PASS**
EC-5	Valid	user@example.com	(empty string)	Validation error: Password required	Zod error returned	**PASS**
EC-6	Valid	user@example.com	abc (3 chars)	Validation error: Password too short	Zod error returned	**PASS**
EC-7	(blank string)	user@example.com	SecurePass1	Validation error: Name required	Zod error returned	**PASS**
EC-8	Valid	duplicate@email.com (exists)	SecurePass1	Error: Email already registered	HTTP 409 returned	**PASS**

Example: Add Job Application — Status Field

The status field accepts only values from the defined pipeline enum.

TC#	Status Input	Expected Result	Actual Result	Status
EC-9	"applied" (valid)	Job created with status applied	Job created	PASS
EC-10	"shortlisted" (valid)	Job created with status shortlisted	Job created	PASS
EC-11	"rejected" (invalid enum)	Validation error: Invalid status	Zod error returned	PASS
EC-12	"" (empty)	Validation error: Status required	Zod error returned	PASS
EC-13	null	Validation error: Status required	Zod error returned	PASS

5.2 Boundary Value Analysis

Example: Task Due Date — Temporal Boundaries

The task system categorises tasks by their due date relative to today. Boundary values for the date dimension are tested.

TC#	Due Date Input	Boundary Type	Expected Result	Actual Result	Status
BVA-1	Yesterday (past)	Below lower active boundary	Appears in Overdue tab	Shown in Overdue	PASS
BVA-2	Today	Lower active boundary (min for Today)	Appears in Today tab and dashboard	Shown in Today + dashboard	PASS
BVA-3	Tomorrow	Just above Today boundary	Appears in Tomorrow tab	Shown in Tomorrow	PASS
BVA-4	7 days from now	Within Upcoming range	Appears in Upcoming tab	Shown in Upcoming	PASS
BVA-5	No due date (null)	Absent boundary	Task created; not shown on dashboard or any tab	Created; no tab entry	PASS
BVA-6	Invalid date string	Non-date input	Validation error: Invalid date	Zod error returned	PASS

Example: OTP Value — Numeric Boundaries

TC#	OTP Input	Boundary Type	Expected Result	Actual Result	Status
BVA-7	6-digit numeric OTP (correct)	Valid exact boundary	Password reset succeeds	Reset succeeded	PASS
BVA-8	5-digit OTP	Below lower boundary	Validation error: Invalid OTP	Error returned	PASS

BVA-9	7-digit OTP	Above upper boundary	Validation error: Invalid OTP	Error returned	**PASS**
BVA-10	Correct OTP submitted after 10 min	Expired temporal boundary	Error: OTP expired	Error returned	**PASS**
BVA-11	Non-numeric string	Invalid class	Validation error: OTP must be numeric	Error returned	**PASS**

5.3 Decision Table Testing

Dashboard Access — Decision Table

Conditions: C1 = User authenticated; C2 = Job data exists in Redis cache; C3 = MongoDB reachable.

Condition / Action	Case 1	Case 2	Case 3	Case 4
C1: User Authenticated	Yes	Yes	No	No
C2: Redis Cache Hit	Yes	No	Yes	No
C3: MongoDB Reachable	Yes	Yes	Yes	No
Expected Outcome	Dashboard from cache	Dashboard from MongoDB; cache populated	Redirect to login	Redirect to login

Add Job Application — Decision Table

Conditions: C1 = User authenticated; C2 = Required fields present and valid; C3 = Status is a valid enum value.

Condition / Action	Case 1	Case 2	Case 3	Case 4
C1: User Authenticated	Yes	Yes	Yes	No
C2: Valid Required Fields	Yes	No	Yes	Yes
C3: Valid Status Enum	Yes	Yes	No	Yes
Expected Outcome	Job saved; cache invalidated	Zod validation error	Zod validation error	401 Unauthorized

6. Fault, Error, and Failure Analysis

This section demonstrates how undetected faults in Taskify could propagate into observable user-facing failures.

Term	Definition
Fault	A static defect in the source code or design — the mistake itself.
Error	The incorrect internal system state caused by executing a fault.
Failure	The externally observable incorrect behaviour experienced by the user.

Chain 1: Missing Cache Invalidation on Job Delete

Error: A developer adds a delete handler for jobs but forgets to call the Redis cache invalidation helper after the Mongoose delete operation.

Fault: The deleteJob controller does not call redisClient.del(userJobsKey) after removing the document.

Trigger: User deletes a job application. The database record is removed, but the cached job list in Redis still contains it.

Failure: The user sees the deleted job still appearing in their job list for up to 5 minutes. This is confusing and misleading — the user may delete the same job again or believe the deletion failed.

Prevention: During code inspection, all three write paths (POST, PATCH, DELETE) were verified to call the cache invalidation helper. This potential fault was confirmed to be absent in the current implementation.

Chain 2: Worker Crash Swallowing OTP Delivery Failure

Error: A developer handles errors in the email worker with a simple console.error() and acknowledges the message to the queue regardless of success or failure.

Fault: emailWorker.js calls channel.ack(msg) even in the catch block, meaning a failed email delivery does not result in requeue or dead-lettering.

Trigger: Nodemailer fails to authenticate with the Gmail SMTP server (e.g., rotated App Password). The worker processes a password-reset OTP message, the email send throws, the error is logged, and the message is acked.

Failure: The user triggers forgot-password, receives no email, has no way to reset their password, and sees no user-facing error — the API returned 200 because the OTP was queued successfully.

Detection: This was identified during formal inspection (inspection item 10). The fix is to call channel.nack(msg, false, true) to requeue on transient failures, and configure a dead-letter queue for persistent failures. Noted as a known gap.

Chain 3: Zod Bypass via Direct API Call

Error: Input validation is only enforced at the route layer, and a developer adds a direct function call to a controller from a new internal route without passing through Zod validation.

Fault: A code path exists to the job creation logic that does not validate the openingType field, allowing an arbitrary string to be stored.

Trigger: A direct API call sends openingType: 'unknown_type' to the unprotected path.

Failure: The database stores an invalid enum value. When the frontend reads this value and tries to render the badge for opening type, it encounters an unrecognised value and either renders nothing or throws a runtime error.

Prevention: Equivalence class testing (EC-11 through EC-13) confirmed that all tested paths through the API correctly reject invalid enum values, confirming Zod validation is uniformly applied on the standard request paths.

7. Software Project Audit

A structured audit of Taskify was conducted against key quality parameters:

Audit Parameter	Score	Remarks
Code Architecture	9/10	Clean MVC separation; controllers, models, routes, services, workers, and jobs are all logically distinct with no observable coupling issues.
Security Foundations	8/10	JWT in HttpOnly cookies, bcrypt 12 rounds, Zod on all inputs, CORS allowlist configured. Missing: rate limiting on OTP endpoint, Helmet.js security headers.
Input Validation	9/10	Zod schemas applied uniformly across all routes. Both syntactic and semantic validation present (enum checks, required fields, type coercion).
Error Handling	8/10	Centralized AppError class and global error middleware are strong. Worker error handling is weak — no dead-letter queue for persistent failures.
Async Architecture	9/10	RabbitMQ decouples email delivery from request-response correctly; cron publishes to queue rather than sending inline — correct design.
Cache Design	9/10	Per-user Redis cache with TTL and complete write-through invalidation. Graceful degradation when Redis is unavailable.
Testing Adequacy	8/10	Thorough black-box testing applied; all test cases passed. No automated tests in the repository — manual testing only.
Documentation Quality	7/10	README is comprehensive with setup, env vars, API overview, and deployment notes. No formal SRS/SDD; no documented API error codes.
Deployment Readiness	9/10	Live deployment on Render + Vercel; environment-based configuration; deployment pitfalls (Render PORT, Upstash TLS, Atlas URL encoding) documented.
Overall	**85/100**	**Production-deployed, well-architected application. Strong async design and validation. Minor gaps in security headers, rate limiting, and worker resilience.**

8. Conclusion

This report successfully demonstrates the practical application of verification and validation techniques — as taught in the Software Testing and Evaluation course up to Chapter 5 — on the Taskify project, a production-deployed full-stack job application tracker.

Static verification through peer review, walkthrough, and formal inspection identified four design ambiguities (cron timezone, OTP TTL index, graceful degradation documentation, logo fallback), one security gap (no rate limiting on the OTP endpoint), and two implementation weaknesses (missing security headers, no dead-letter queue for worker failures). All observations were documented, and the majority were confirmed to be absent or non-critical in the actual implementation.

Validation through equivalence class testing, boundary value analysis, and decision table testing covered all major input paths across user registration, job tracking, task management, and password reset flows. All 25+ test cases across the three techniques produced the expected results, confirming that Taskify correctly handles both valid inputs and edge-case invalid inputs.

The fault-error-failure analysis demonstrated three concrete propagation chains: cache invalidation on delete, worker silent swallow of delivery failures, and Zod bypass via internal call paths. The first and third were confirmed mitigated in the current implementation; the second was identified as a known gap requiring a dead-letter queue configuration.

Taskify demonstrates how modern Node.js full-stack applications can be built with strong separation of concerns, async-first architecture, and input validation discipline. The V&V techniques applied in this report confirm that proactive, systematic verification significantly reduces downstream defects and that the application meets all stated functional requirements in its deployed state.

References

- Singh, Y. (2012). Software Testing. Cambridge University Press India.
- Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- Myers, G. J., Badgett, T., & Sandler, C. (2011). The Art of Software Testing (3rd ed.). John Wiley & Sons.
- IEEE Standard 1012-2016. IEEE Standard for System, Software, and Hardware Verification and Validation.
- Taskify Project Repository. (2026). https://github.com/npxpatel/Taskify
- MongoDB Documentation. (2024). https://www.mongodb.com/docs/
- RabbitMQ Documentation. (2024). https://www.rabbitmq.com/documentation.html